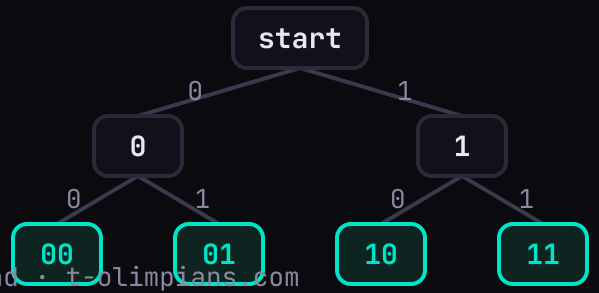
// T-OLIMPIANS — PREZENTACJA DO LEKCJI

Generowanie wszystkich możliwości

Rekurencja jako „pętla w pętli” o dowolnej głębokości: ciągi, podzbiory (bierz / nie bierz), kombinacje, permutacje i nawiasy. Drzewo decyzji, cofanie wyboru i trzy wzorce: wypisz wszystkie, znajdź jedno, policz.

C++ · REKURENCJA · CZĘŚĆ 4 Z 6 · PO CZĘŚCIACH 1-3



PLAN PREZENTACJI

01 Zadania „wypisz wszystko”
02 Ciągi zero-jedynkowe
03 Drzewo decyzji
04 Więcej cyfr, mniej gałęzi
05 Podzbiory: bierz / nie bierz
06 Cofanie wyboru
07 Wypisz, znajdź, policz
08 Kombinacje: k z n
09 Permutacje
10 Nawiasy i rozkłady
11 Powtórzenia w danych
12 Ile tego jest?
13 Sprawdź się
14 Podsumowanie

01 Nowy rodzaj zadań: wypisz wszystkie możliwości

W wielu zadaniach trzeba przejrzeć **wszystkie** możliwości — i wypisać je, policzyć albo znaleźć najlepszą:

`wszystkie kody PIN`
z cyfr 1, 2, 3, długości 4 — jakie są?

`wszystkie drużyny`
które 3 osoby z 6 można wysłać na zawody?

`wszystkie kolejności`
w jakiej kolejności odwiedzić 5 miast?

`wszystkie podzbiory`
które przedmioty spakować, żeby suma wag była równa 20?

Wszystkie te zadania rozwiązuje ten sam pomysł: **rekurencja, która podejmuje decyzje jedna po drugiej**.

01 Dlaczego pętle nie wystarczą?

CIĄGI ZERO-JEDYNKOWE DŁUGOŚCI 3

```
for (int a = 0; a <= 1; a++)
    for (int b = 0; b <= 1; b++)
        for (int c = 0; c <= 1; c++)
            cout << a << b << c << endl;
```

NA EKRANIE

```
000
001
010
011
100
101
110
111
```

Dla długości 3 potrzebujemy **3 pętli** jedna w drugiej. Dla długości 4 — czterech. A jeśli długość **n** wczytujemy z wejścia?

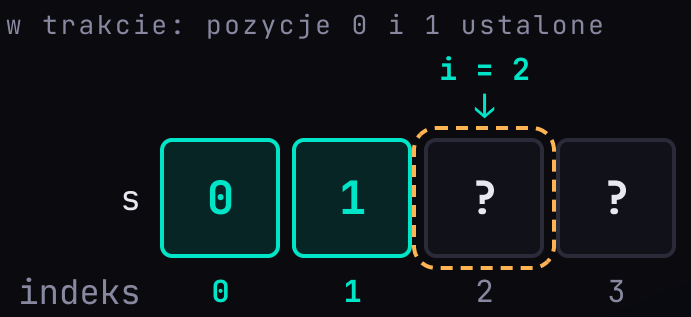
Uwaga: Nie da się napisać „n pętli w sobie”, gdy n znamy dopiero w trakcie działania programu.

Rekurencja to właśnie **pętla w pętli w pętli…** o dowolnej głębokości: każde wywołanie to jedna pętla, a głębokość rekurencji = liczba pętli.

02 Pomysł: wypełniaj ciąg pozycja po pozycji

Funkcja `generuj(i)` zajmuje się **pozycją i** ciągu. Pozycje 0, …, i − 1 są już ustalone przez wcześniejsze wywołania.

1. wstaw na pozycję i **0** i każ wywołaniu generuj(i + 1) dokończyć ciąg na wszystkie sposoby,
2. potem wstaw **1** i znowu wywołaj generuj(i + 1),
3. gdy i == n — wszystkie pozycje są ustalone: **wypisz** ciąg.



02 Ciągi zero-jedynkowe — kod

```
int n;
string s;

void generuj(int i) {
    if (i == n) {
        cout << s << endl;
        return;
    }
    s[i] = '0';
    generuj(i + 1);
    s[i] = '1';
    generuj(i + 1);
}
```

N = 3, W MAIN: S = STRING(N, '0'); GENERUJ(0);

```
000
001
010
011
100
101
110
111
```

Każde wywołanie ma **dwa** wywołania w głąb — jak fib w części 3. Ale tym razem nic się nie powtarza: każdy liść to **inny** ciąg.

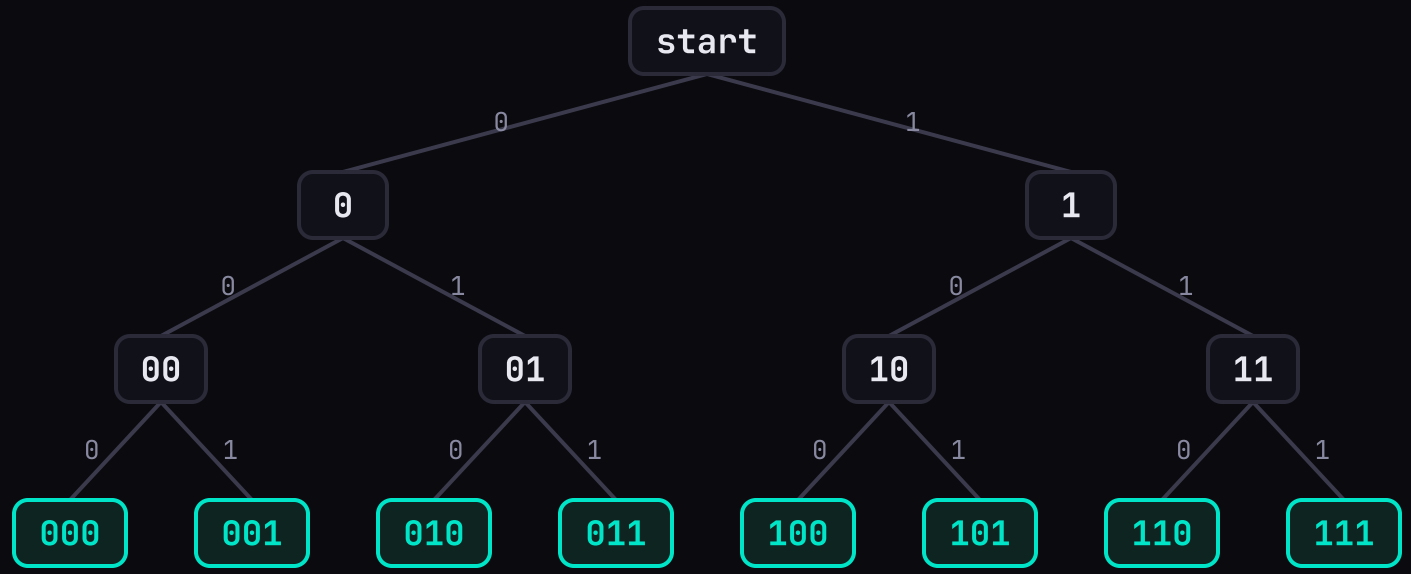
02 Śledzimy n = 2

↓ generuj(0): s[0] = 0
  ↓ generuj(1): s[1] = 0
    ▸ generuj(2): wypisuje **00**
  ↓ generuj(1): s[1] = 1
    ▸ generuj(2): wypisuje **01**
↓ generuj(0): s[0] = 1
  ↓ generuj(1): s[1] = 0
    ▸ generuj(2): wypisuje **10**
  ↓ generuj(1): s[1] = 1
    ▸ generuj(2): wypisuje **11**

Zwróć uwagę: s[1] jest **nadpisywane** — najpierw 0, potem 1. Po powrocie do generuj(0) i wstawieniu s[0] = 1 stara wartość s[1] nie przeszkadza, bo generuj(1) i tak ją nadpisze.

> Dzięki temu, że 0 próbujemy **przed** 1, ciągi wychodzą w kolejności alfabetycznej.

03 Drzewo decyzji dla n = 3



Drzewo wywołań to tu **drzewo decyzji**: na każdym poziomie decydujemy o jednej pozycji (podpis krawędzi). Każda droga od korzenia do liścia to **jeden ciąg** — liści jest 2 · 2 · 2 = **8**.

03 Ile jest liści, ile wywołań?

n	ciągów (liści)	wywołań
1	2	3
3	8	15
10	1 024	2 047
20	ok. miliona	ok. 2 milionów
30	ok. miliarda	**za dużo**

Każdy poziom ma **2 razy** więcej węzłów. Ciągów długości n jest $\mathbf{2^n}$, a wywołań — niecałe dwa razy więcej.

Uwaga: Memoizacja tu nie pomoże: wszystkie liście są **różne** i każdy trzeba wypisać. Dlatego w zadaniach „wypisz wszystko" n jest zwykle małe — do ok. 20.

04 Więcej możliwości na pozycji: cyfry od 1 do k

```
void generuj(int i) {
    if (i == n) {
        cout << s << endl;
        return;
    }
    for (char c = '1'; c < '1' + k; c++) {
        s[i] = c;
        generuj(i + 1);
    }
}
```

Na każdej pozycji jest teraz **k** możliwości — zamiast pisać k wywołań, piszemy **pętlę**, a w niej jedno wywołanie.

N = 2, K = 3

```
11
12
13
21
22
23
31
32
33
```

Ciągów jest k^n: k możliwości na każdej z n pozycji.

04 Ograniczenie: bez dwóch jedynek obok siebie

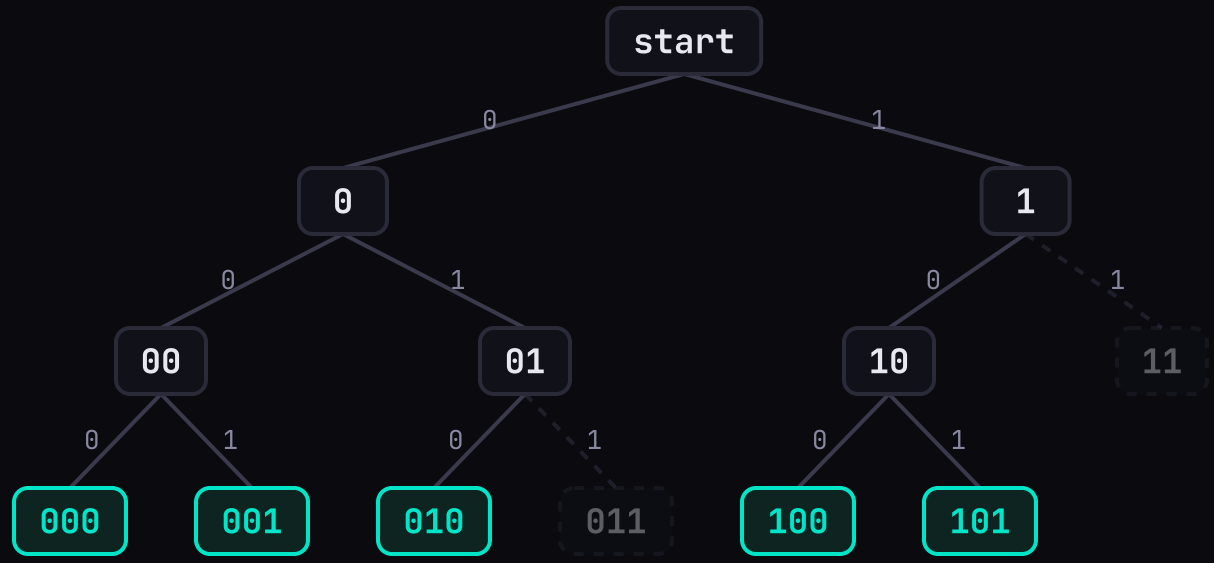
```
void generuj(int i) {
    if (i == n) {
        cout << s << endl;
        return;
    }
    s[i] = '0';
    generuj(i + 1);
    if (i == 0 || s[i - 1] == '0') {
        s[i] = '1';
        generuj(i + 1);
    }
}
```

Jedynkę wolno wstawić tylko wtedy, gdy przed nią **nie ma jedynki**. Sprawdzamy to od razu — zanim zejdziemy głębiej.

N = 3

```
000
001
010
100
101
```

04 Obcinanie gałęzi



Wyblakłe węzły to gałęzie, do których **w ogóle nie wchodzimy** — skoro „11” już jest złe, nie ma sensu dopisywać do niego czegokolwiek. Im wcześniej odetniemy złą gałąź, tym mniej pracy.

05 Podzbiory = ciągi zero-jedynkowe

ciąg	podzbiór {1, 2, 3}
000	{ } (pusty)
001	{3}
010	{2}
011	{2, 3}
100	{1}
101	{1, 3}
110	{1, 2}
111	{1, 2, 3}

Podzbiór zbioru {1, …, n} to **n decyzji**: czy liczba 1 jest w podzbiorze? czy 2 jest? … Każda decyzja to 0 (**nie bierz**) albo 1 (**bierz**).

Każdy podzbiór odpowiada więc dokładnie jednemu ciągowi zero-jedynkowemu — i podzbiorów jest $\mathbf{2^n}$.

> Ten pomysł nazywa się po angielsku *pick / not pick* — u Strivera zobaczysz go w filmie L6.

05 Podzbiory — kod

```
int n;
vector<int> wybrane;

void podzbiory(int i) {
    if (i > n) {
        for (int x : wybrane) cout << x << " ";
        cout << endl;
        return;
    }
    podzbiory(i + 1);          // nie bierz i
    wybrane.push_back(i);
    podzbiory(i + 1);          // bierz i
    wybrane.pop_back();
}
```

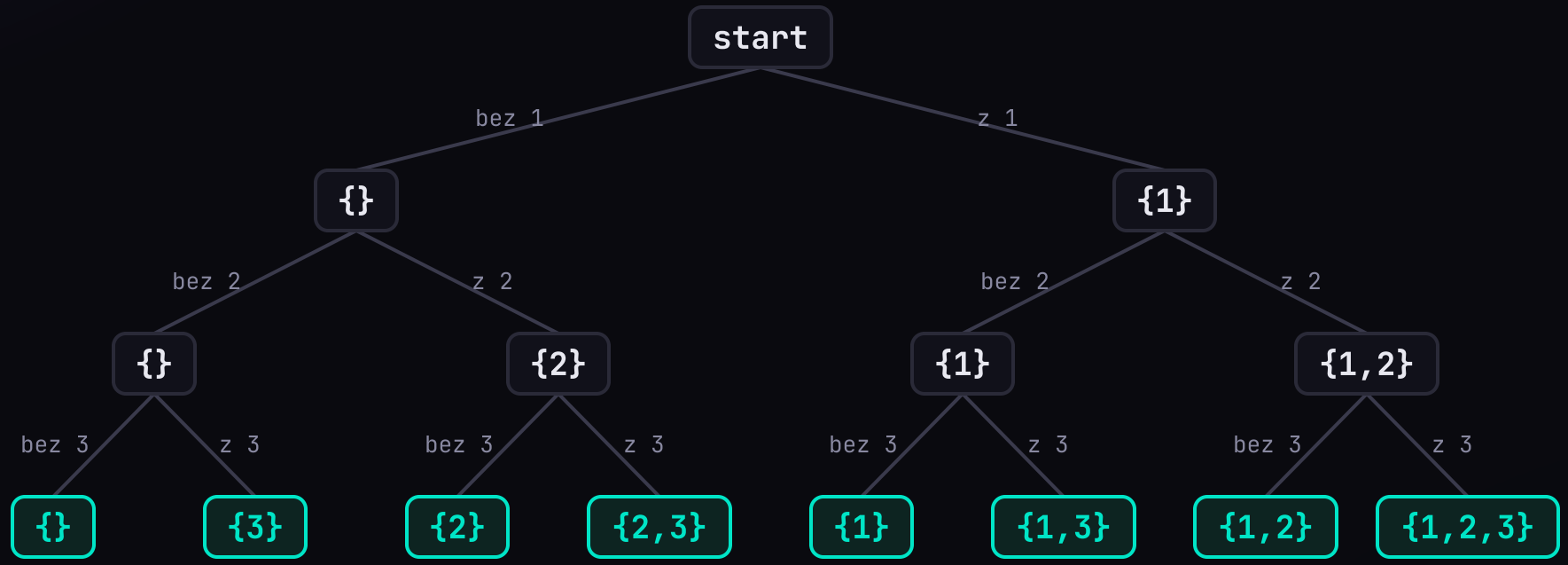
Wybrane liczby trzymamy w **vector**. Decyzja „bierz" = dopisz liczbę na koniec wektora, zejdź głębiej, a po powrocie **usuń ją**.

PODZBIORY(1) DLA N = 3

```
3
2
2 3
1
1 3
1 2
1 2 3
```

Pierwszy wiersz jest pusty — to podzbiór pusty.

05 Drzewo decyzji: bierz / nie bierz



Na poziomie i decydujemy o liczbie i: lewa krawędź — **bez niej**, prawa — **z nią**. W liściach — gotowe podzbiory (od lewej: kolejność wypisywania).

06 Po co pop_back?

```
podzbiory(i + 1);
wybrane.push_back(i);
podzbiory(i + 1);
wybrane.pop_back();    // <- usuń i
```

Wektor **wybrane** jest **jeden** — wspólny dla wszystkich wywołań (to zmienna globalna). Gdy wracamy z gałęzi „bierz i", liczba i musi z niego zniknąć, bo w innych gałęziach jej nie wybraliśmy.

Z POP_BACK — DOBRZE

```
3
2
2 3
1
1 3
1 2
1 2 3
```

BEZ POP_BACK — ŹLE

```
3
3 2
3 2 3
3 2 3 1
…
```

06 Wektor wybrane krok po kroku (n = 2)

- ↓ nie bierz 1 → `{ }`
  - ▸ nie bierz 2 → wypisz `{ }`
  - ↓ push 2 → `{2}`, wypisz **2**
  - ↑ pop → `{ }`
- ↓ push 1 → `{1}`
  - ▸ nie bierz 2 → wypisz **1**
  - ↓ push 2 → `{1, 2}`, wypisz **1 2**
  - ↑ pop → `{1}`
- ↑ pop → `{ }`

Po każdym **push_back** jest dokładnie jedno **pop_back**. Dzięki temu po powrocie z wywołania wektor wygląda **tak samo** jak przed nim.

> Na końcu całej rekurencji wektor jest znowu pusty — wszystko, co dopisaliśmy, zostało cofnięte.

06 Schemat: wybierz → zejdź → cofnij

1 · wybierz

zmień stan: `wybrane.push_back(x)`, `uzyta[x] = true`, `s[i] = c` …

2 · zejdź

wywołaj rekurencję dla następnej decyzji: `generuj(i + 1)`

3 · cofnij

przywróć stan sprzed wyboru: `pop_back()`, `uzyta[x] = false`

To jest serce backtrackingu (części 5). Każdą zmianę wspólnego stanu trzeba po powrocie odwrócić.

Wyjątek: gdy stan jest **parametrem** (np. suma s w `f(i + 1, s + a[i])`), każde wywołanie ma własną kopię i niczego nie trzeba cofać.

07 Trzy wzorce rekurencji generującej

`wypisz wszystkie`

Funkcja **void**. W liściu (gdy decyzje są podjęte) — sprawdź i **wypisz**. Przejdzie całe drzewo.

`znajdź jedno`

Funkcja **bool**. W liściu zwróć true, gdy pasuje. Jeśli wywołanie zwróciło true — **od razu** zwróć true dalej, bez sprawdzania reszty drzewa.

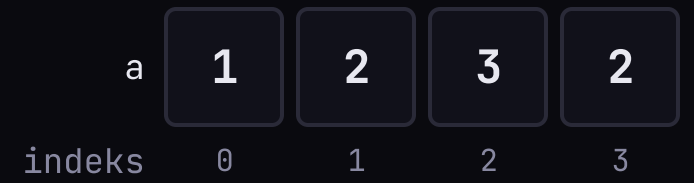
`policz`

Funkcja **int / long long**. Liść zwraca 1 (pasuje) albo 0. Węzeł zwraca **sumę** wyników swoich dzieci.

Ten podział Striver omawia w filmie L7 („Print all / Print one / Count"). Zobaczmy wszystkie trzy na jednym zadaniu: **podzbiory tablicy o sumie k**.

07 Wzorzec 1: wypisz wszystkie podzbiory o sumie k

```
void wypisz(int i, int s) {
    if (i == n) {
        if (s == k) {
            for (int x : wybrane) cout << x << " ";
            cout << endl;
        }
        return;
    }
    wypisz(i + 1, s);
    wybrane.push_back(a[i]);
    wypisz(i + 1, s + a[i]);
    wybrane.pop_back();
}
```



K = 5

```
3 2
2 3
1 2 2
```

Suma **s** jest parametrem — każde wywołanie ma swoją. Wektor **wybrane** jest wspólny — trzeba go cofać.

07 Wzorzec 2: znajdź jeden (czy w ogóle się da?)

```
bool da(int i, int s) {
    if (i == n) return s == k;
    if (da(i + 1, s + a[i])) return true;
    if (da(i + 1, s)) return true;
    return false;
}
```

Gdy którakolwiek gałąź zwróci **true**, przekazujemy true w górę i **nie zaglądamy** do pozostałych gałęzi.

> W najgorszym razie (gdy odpowiedź to „nie”) i tak przejdziemy całe drzewo — ale gdy odpowiedź to „tak”, często kończymy bardzo wcześnie.

07 Wzorzec 3: policz

```
int ile(int i, int s) {
    if (i == n) return s == k ? 1 : 0;
    return ile(i + 1, s) + ile(i + 1, s + a[i]);
}
```

Każdy liść zwraca **1** (podzbiór pasuje) albo **0**. Każdy węzeł dodaje wyniki swoich dwóch dzieci. W korzeniu wychodzi liczba wszystkich pasujących podzbiorów.

```
A = 1 2 3 2, K = 5
```

```
3
```

Gdy liczby są dodatnie, dodaj obcinanie: **if (s > k) return 0;** — suma już nie zmaleje.

07 Sumy wszystkich podzbiorów — bez wektora

```
void sumy(int i, int s) {
    if (i == n) {
        wynik.push_back(s);
        return;
    }
    sumy(i + 1, s);
    sumy(i + 1, s + a[i]);
}
```

Gdy interesuje nas tylko **suma** wybranych liczb, nie trzeba pamiętać, które to były — wystarczy nieść sumę w parametrze.

A = 2 3 5, PO SORTOWANIU WYNIKÓW

0 2 3 5 5 7 8 10

08 Kombinacje: wybierz k liczb z n

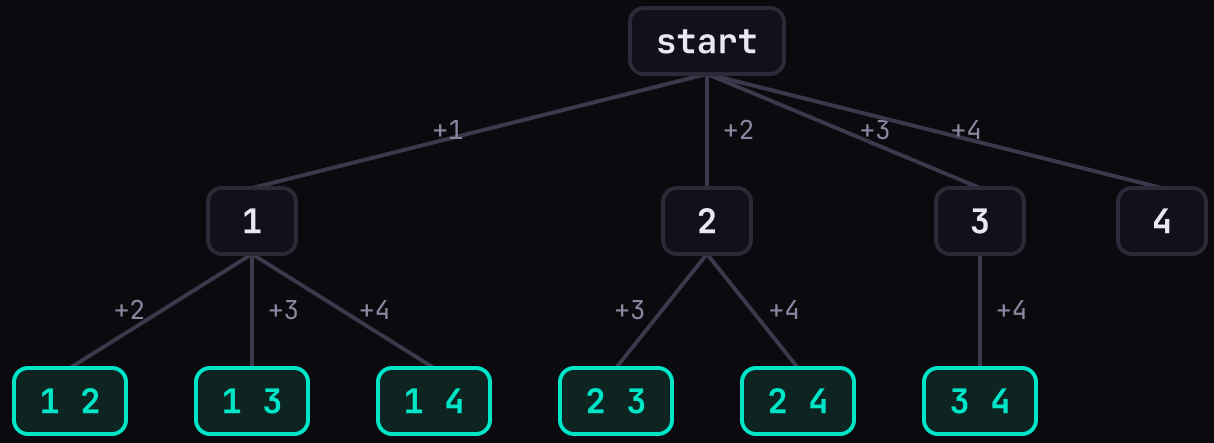
```
void wybieraj(int od) {
    if (wybrane.size() == k) {
        for (int x : wybrane) cout << x << " ";
        cout << endl;
        return;
    }
    for (int x = od; x <= n; x++) {
        wybrane.push_back(x);
        wybieraj(x + 1);
        wybrane.pop_back();
    }
}
```

Tym razem nie decydujemy o każdej liczbie „tak/nie”, tylko **dobieramy kolejną liczbę** — w pętli, spośród liczb od **od** do n.

N = 4, K = 2: WYBIERAJ(1)

```
1 2
1 3
1 4
2 3
2 4
3 4
```

08 Drzewo kombinacji n = 4, k = 2



Następna liczba jest zawsze **większa** od poprzedniej (wołamy `wybieraj(x + 1)`), więc „1 2" powstaje, a „2 1" — nigdy. Każdy zbiór pojawia się **dokładnie raz**, od razu z liczbami rosnąco.

08 Ile jest kombinacji?

n	k	C(n, k)
4	2	6
5	3	10
10	5	252
20	10	184 756
30	15	ok. 155 milionów

To ten sam symbol Newtona C(n, k), który w części 3 **liczyliśmy** trójkątem Pascala — tym razem każdą kombinację **wypisujemy**.

> Obcinanie: jeśli liczb do końca (n − x + 1) jest za mało, żeby dobrać brakujące — można przerwać pętlę. Np. dla k = 2 i n = 4 nie ma sensu zaczynać od x = 4.

09 Permutacje — wszystkie kolejności

Permutacja liczb 1, …, n to ustawienie ich wszystkich w jakiejś kolejności. Dla n = 3: **123, 132, 213, 231, 312, 321**.

n	permutacji = n!
3	6
5	120
8	40 320
10	3 628 800
12	ok. 479 milionów

Pomysł: na pozycję i wstawiamy **każdą liczbę, która jeszcze nie została użyta**. Do zapamiętania, które są zajęte, służy tablica **uzyta[]**.

> Na pierwszą pozycję jest n możliwości, na drugą n − 1, … — razem n · (n − 1) · … · 1 = **n!**.

09 Permutacje — kod

```
int n, perm[10];
bool uzyta[10];

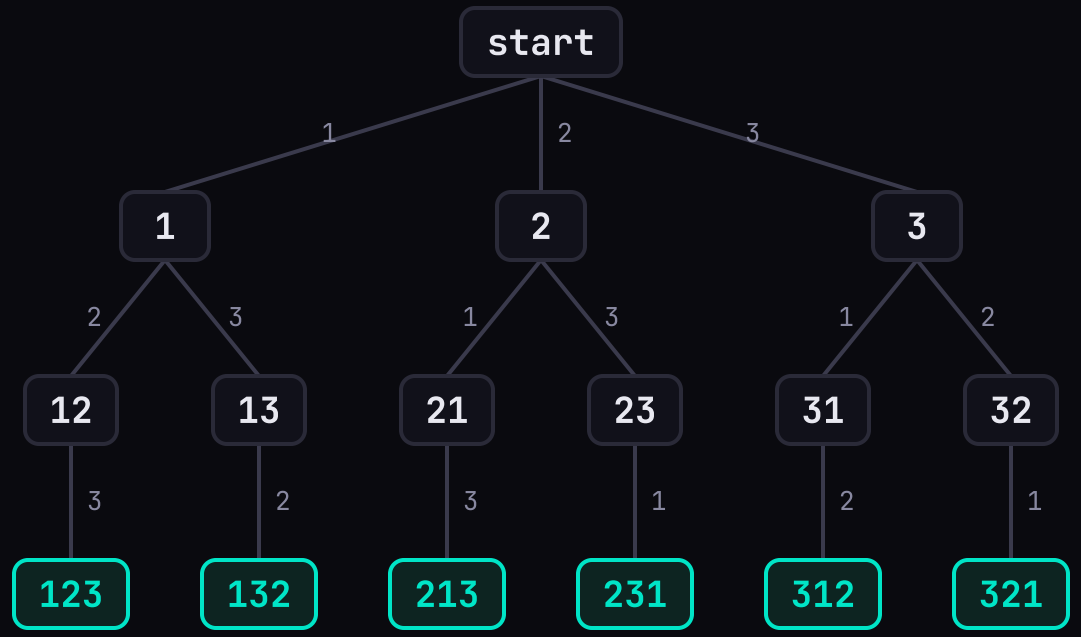
void permutuj(int i) {
    if (i == n) {
        for (int j = 0; j < n; j++) cout << perm[j] << " ";
        cout << endl;
        return;
    }
    for (int x = 1; x <= n; x++) {
        if (uzyta[x]) continue;
        uzyta[x] = true;
        perm[i] = x;
        permutuj(i + 1);
        uzyta[x] = false;
    }
}
```

1. `if (uzyta[x]) continue;` — liczba już stoi wcześniej, pomijamy ją,
2. **wybierz**: zaznacz x jako użytą i wstaw na pozycję i,
3. **zejdź**: `permutuj(i + 1)`,
4. **cofnij**: `uzyta[x] = false` — x znowu wolna dla innych gałęzi.

N = 3

```
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1
```

09 Drzewo permutacji n = 3



Na pierwszym poziomie 3 gałęzie, na drugim po 2, na trzecim po 1 — razem **3! = 6** liści, od razu w kolejności leksykograficznej (bo pętla idzie od x = 1 w górę).

09 Druga metoda: zamiany (swap)

```
void permutuj(int i) {
    if (i == n) { wypisz(); return; }
    for (int j = i; j < n; j++) {
        swap(a[i], a[j]);
        permutuj(i + 1);
        swap(a[i], a[j]);
    }
}
```

Na pozycję i wstawiamy po kolei każdy element z pozycji i, …, n − 1 — zamieniając go miejscami. Po powrocie **zamieniamy z powrotem** (cofnięcie wyboru).

Tę metodę Striver pokazuje w filmie L13. Działa bez tablicy uzyta[], ale permutacje **nie wychodzą** w kolejności leksykograficznej — gdy zadanie jej wymaga, użyj pierwszej metody.

10 Poprawne nawiasowania

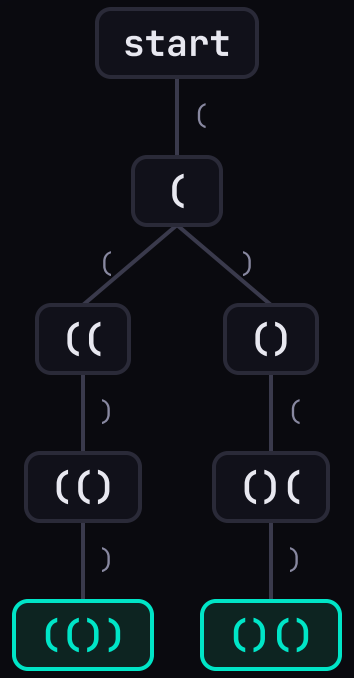
Napis z n nawiasów „(" i n nawiasów „)" jest poprawny, gdy każdy nawias ma swoją parę. Dla n = 3: **((()))**, **(()())**, **(())()**, **()(())**, **()()()**.

Dopisujemy po jednym nawiasie, ale **tylko taki, który ma sens**:

1. „(" — gdy otworzyliśmy mniej niż n: `otw < n`
2. „)" — gdy jest co zamykać: `zam < otw`

```
void generuj(int otw, int zam) {
    if (zam == n) { cout << s << endl; return; }
    if (otw < n) {
        s.push_back('(');
        generuj(otw + 1, zam);
        s.pop_back();
    }
    if (zam < otw) {
        s.push_back(')');
        generuj(otw, zam + 1);
        s.pop_back();
    }
}
```

10 Drzewo dla n = 2



Z „start” wolno tylko otworzyć. Z „((” wolno już tylko zamykać. Gałęzie prowadzące do złych napisów (jak „)(”) **w ogóle nie powstają**.

> Poprawnych nawiasowań z n par jest: 1, 2, 5, 14, 42, 132, 429, 1430… — to tak zwane **liczby Catalana**.

10 Rozkłady liczby — wypisz wszystkie

```
void rozklad(int zostalo, int maks) {
    if (zostalo == 0) { wypisz(); return; }
    for (int x = min(zostalo, maks); x >= 1; x--) {
        skladniki.push_back(x);
        rozklad(zostalo - x, x);
        skladniki.pop_back();
    }
}
```

W części 3 **liczyliśmy** rozkłady liczby n. Teraz każdy **wypisujemy**. Kolejny składnik nie może być większy od poprzedniego — dzięki temu 3+1 i 1+3 nie powstaną oba.

ROZKLAD(4, 4)

```
4
3+1
2+2
2+1+1
1+1+1+1
```

11 Problem: powtórzenia w danych

a	1	2	2
indeks	0	1	2

Podzbiory tej tablicy, wybierane po pozycjach, to m.in. {1, 2} z pierwszą dwójką i {1, 2} z drugą dwójką. Na ekranie wyglądają **tak samo**!

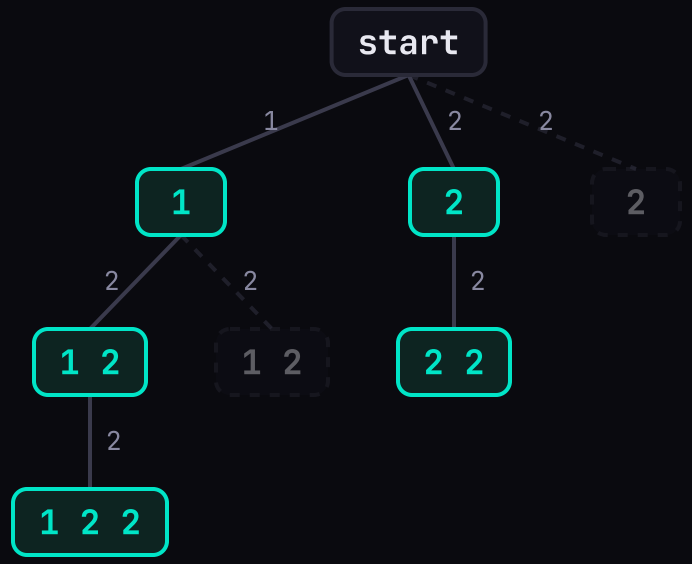
> **Uwaga:** Gdy zadanie każe wypisać **różne** podzbiory (albo różne kombinacje, permutacje), takie powtórki trzeba wyeliminować.

Pomysł (Striver, film L11):

1. **posortuj** tablicę — równe liczby staną obok siebie,
2. gdy w pętli wybierasz **kolejny element**, a ten jest równy poprzedniemu na tym samym poziomie — **pomiń go**.

```
for (int i = od; i < n; i++) {
    if (i > od && a[i] == a[i - 1]) continue;
    ...
}
```

11 Dlaczego `i > od`?



Na **jednym poziomie** (w jednej pętli) z kilku równych liczb próbujemy tylko **pierwszej** — wyblakłe gałęzie to pominięte powtórki.

Ale **głębiej** druga dwójka wolno wziąć — stąd „2 2” i „1 2 2”. Warunek **i > od** pilnuje, żeby pomijać tylko powtórki w tej samej pętli, a nie pierwszy element pętli.

11 Permutacje słowa z powtórzonymi literami

```
int ile[26];       // ile zostało każdej litery

void generuj(int i) {
    if (i == n) { cout << slowo << endl; return; }
    for (int c = 0; c < 26; c++) {
        if (ile[c] == 0) continue;
        ile[c]--;
        slowo[i] = 'a' + c;
        generuj(i + 1);
        ile[c]++;
    }
}
```

Z „aab” przestawianiem liter dostaniemy tylko **3** różne słowa: aab, aba, baa. Zamiast wybierać **pozycję** litery, wybieramy **literę** — każdą różną literę raz.

SŁOWO „AAB”

aab
aba
baa

Pętla po literach a…z daje od razu kolejność alfabetyczną.

12 Ile tego jest? — rozpoznaj złożoność

Co generujemy	Ile wyników	Rozsądne n (ok. 1 s)
ciągi zero-jedynkowe, podzbiory	2^n	$n \le 20–25$
ciągi z k znaków	k^n	zależy od k: $3^n \to n \le 15$
kombinacje k z n	C(n, k)	$n \le 30$ dla małych k
permutacje	n!	$n \le 10–11$
nawiasowania	liczby Catalana	$n \le 15$

Rekurencja generująca działa tak długo, jak duże jest drzewo — a drzewo ma co najmniej tyle liści, ile wyników. Gdy w zadaniu n jest małe (≤ 20), to często znak, że **wolno sprawdzić wszystko**.

13 Pytanie 1: co wypisze `generuj(0)` dla `n = 2`?

```
void generuj(int i) {
    if (i == n) { cout << s << endl; return; }
    s[i] = 'b'; generuj(i + 1);
    s[i] = 'a'; generuj(i + 1);
}
```

NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 1: bb ba ab aa

NA EKRANIE

```
bb
ba
ab
aa
```

Najpierw próbujemy **b**, potem **a** — więc kolejność jest odwrotna niż alfabetyczna. Kolejność wywołań w funkcji decyduje o kolejności wyników!

13 Pytanie 2: ile wierszy wypisze `wybieraj(1)` dla `n = 5, k = 3`?

```
void wybieraj(int od) {
    if (wybrane.size() == k) {
        for (int x : wybrane) cout << x << " ";
        cout << endl;
        return;
    }
    for (int x = od; x <= n; x++) {
        wybrane.push_back(x);
        wybieraj(x + 1);
        wybrane.pop_back();
    }
}
```

NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 2: 10

NA EKRANIE

```
1 2 3
1 2 4
1 2 5
1 3 4
1 3 5
1 4 5
2 3 4
2 3 5
2 4 5
3 4 5
```

Tyle, ile jest 3-elementowych podzbiorów zbioru 5-elementowego: C(5, 3) = 10.

13 Pytanie 3: co tu jest źle?

```
void permutuj(int i) {
    if (i == n) { wypisz(); return; }
    for (int x = 1; x <= n; x++) {
        if (uzyta[x]) continue;
        uzyta[x] = true;
        perm[i] = x;
        permutuj(i + 1);
    }
}
```

NA EKRANIE

?

Program dla n = 3 wypisuje tylko jedną permutację.

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 3: brak cofnięcia

POPRAWNIE

```
uzyta[x] = true;
perm[i] = x;
permutuj(i + 1);
uzyta[x] = false;
```

Uwaga: Bez **uzyta[x] = false** po pierwszej permutacji 1 2 3 wszystkie liczby są oznaczone jako zajęte już na zawsze — żadna inna gałąź nie znajdzie wolnej liczby. Zawsze cofaj zmiany wspólnego stanu.

13 Pytanie 4: ile razy wywoła się `podzbiory` dla `n = 3`?

```
int n;
vector<int> wybrane;

void podzbiory(int i) {
    if (i > n) {
        for (int x : wybrane) cout << x << " ";
        cout << endl;
        return;
    }
    podzbiory(i + 1);          // nie bierz i
    wybrane.push_back(i);
    podzbiory(i + 1);          // bierz i
    wybrane.pop_back();
}
```

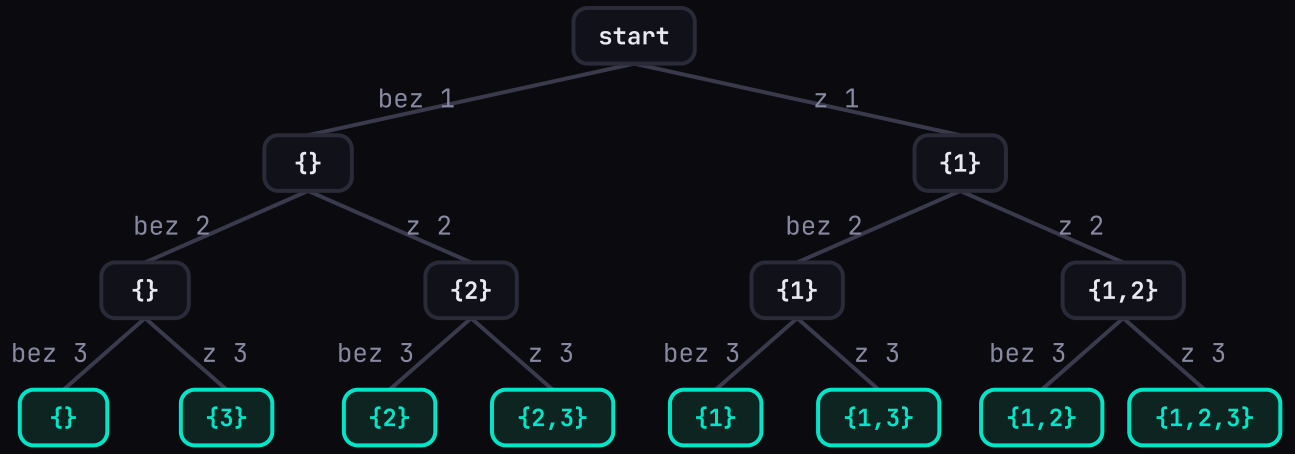
NA EKRANIE

?

Łącznie z pierwszym wywołaniem podzbiory(1).

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 4: 15



Drzewo ma 1 + 2 + 4 + 8 = **15** węzłów: 8 liści (podzbiorów) i 7 węzłów, w których podejmujemy decyzje.

14 Ściąga: generowanie wszystkich możliwości

Co	Rdzeń kodu	Uwagi
ciągi	`for (c : znaki) { s[i] = c; f(i + 1); }`	i == n → wypisz
podzbiory	`f(i + 1); push(i); f(i + 1); pop();`	bierz / nie bierz
kombinacje	`for (x = od..n) { push(x); f(x + 1); pop(); }`	następna liczba większa
permutacje	`if (!uzyta[x]) { uzyta = 1; f(i + 1); uzyta = 0; }`	tablica uzyta[]
powtórzenia	`if (i > od && a[i] == a[i - 1]) continue;`	najpierw sort
wypisz / znajdź / policz	void / bool / int	policz: liść zwraca 1 lub 0
cofanie	każda zmiana wspólnego stanu → odwróć po powrocie	parametry cofają się same

14 Zadania na koniec — Twoja kolej

Zadania są na **t-olimpians.com → Kompendium → Funkcje rekurencyjne → Krok 4**. Każde ma sprawdzarkę i wskazówkę:

1. Wszystkie ciągi zero-jedynkowe
2. Ciągi z cyfr od 1 do k
3. Bez dwóch jedynek obok siebie
4. Wszystkie podzbiory
5. Sumy wszystkich podzbiorów
6. Ile podzbiorów ma sumę k?
7. Czy da się uzbierać k?
8. Wybierz k z n
9. Wszystkie permutacje
10. Permutacje napisu
11. Poprawne nawiasowania
12. Rozkłady liczby na składniki
13. Suma z powtórzeniami
14. Każda liczba co najwyżej raz
15. Podzbiory bez powtórzeń
16. Klawiatura telefonu
17. K-ta permutacja

// KONIEC CZĘŚCI 4

Umiesz już przejrzeć wszystkie możliwości!

Każda decyzja to poziom drzewa, każdy liść to jeden wynik. Wybierz, zejdź, cofnij — i pamiętaj, ile liści ma drzewo. Zrób zadania z kroku 4, a potem część 5: backtracking, czyli szukanie z wycofywaniem.

